

Team Task Tracker API

Design & Technical Reference — database design, container setup, API documentation, deliberate tradeoffs, and a roadmap of future improvements.

Backend: Node.js · TypeScript · Express · Prisma · MySQL 8 · Redis · Zod · Swagger · Jest · Docker

Frontend (bonus): React 18 · TypeScript · Vite · Redux Toolkit + RTK Query · Tailwind CSS

Highlights: JWT + refresh-token rotation · middleware-level RBAC · Redis cache per assignee · real-time SSE · analytics

Live deployment

Live app	https://tasktracker.saitejareddy.online
API + Swagger docs	https://task.saitejareddy.online/docs
Health check	https://task.saitejareddy.online/health
GitHub	https://github.com/mintureddy25/team-task-tracker

Demo login — ADMIN admin@saiteja.com · MANAGER priya@saiteja.com · MEMBER sneha@saiteja.com · password [Saiteja@2026](#)

1 • Database Design

MySQL 8 accessed through the Prisma ORM. The schema is multi-tenant: an `organization` is the tenancy boundary and every record hangs off an org.

Models (6)

Model	Purpose
<code>organizations</code>	Tenancy boundary — the root of every tenant's data.
<code>users</code>	<code>(orgId, email)</code> unique <i>per org</i> , so the same person can exist in multiple orgs. Holds role (ADMIN / MANAGER / MEMBER).
<code>projects</code>	Owned by an org; cascade-deletes to its tasks.
<code>tasks</code>	title, description, priority, status, nullable assignee, due date.
<code>refresh_tokens</code>	SHA-256 hashed with a rotation chain via <code>replacedBy</code> (reuse detection).
<code>notifications</code>	Persisted, with a JSON <code>payload</code> for type flexibility.

Indexes

Indexes target the spec's frequently-queried fields plus the hottest composite paths:

Table	Index	Serves
tasks	<code>@@index([status])</code>	filter by status
tasks	<code>@@index([assigneeId])</code>	filter by assignee
tasks	<code>@@index([dueDate])</code>	overdue / due-soon queries
tasks	<code>@@index([assigneeId, status])</code>	"my open tasks" board (hot path)
tasks	<code>@@index([projectId, status])</code>	project-scoped boards
users	<code>@@unique([orgId, email])</code>	per-org uniqueness

Design decisions explained

1. UUID primary keys (CHAR(36)) instead of INT AUTO_INCREMENT. Removes IDOR risk (you can't guess `/tasks/2` to reach another org's task), keeps multi-tenant data merge-safe, and matches modern API conventions.

2. Composite index `(assigneeId, status)` on tasks. The single hottest query is a MEMBER's board — "my open tasks". A single-column index on `assigneeId` would still scan all of their rows to filter by status; the composite turns it into an index-only seek. Paired with `(projectId, status)` for project boards.

3. Refresh tokens stored as SHA-256 hashes with a rotation chain. A DB leak doesn't hand attackers usable tokens (only hashes), and `replacedBy` enables reuse detection: if a revoked refresh token is presented, the entire chain for that user is revoked — forcing re-login on all devices. Standard OWASP defence against stolen tokens.

4. **Notification.payload** is **JSON, not normalized columns**. Notification types will grow (mentions, comments, due-soon). JSON lets each type carry its own shape without N nullable columns. Tradeoff: can't query inside payload efficiently — acceptable since notifications are append-only and rarely filtered by content.
5. **assigneeId** nullable with **onDelete: SET NULL**. Deleting a user must not cascade-delete their task history. Tasks survive as unassigned and audit / who-did-what queries still work.

Caching (per-assignee). Key shape `tasks:assignee:{assigneeId}:{md5(query)}`. Only assignee-filtered `GET /tasks` reads are cached (the spec's hot path), 60s TTL. On any task mutation the affected assignee's namespace is invalidated by `SCAN MATCH` prefix — `PATCH` wipes both old *and* new assignee to cover reassignment. Cache is read-shedding, not source of truth, so worst-case staleness is bounded by the TTL.

2 • Running with Docker

The stack is fully self-contained. One command, **no `.env` and no manual setup**:

```
docker compose up --build
```

Three services boot together:

Service	Port	Purpose
<code>api</code>	3000	Express API — runs <code>prisma migrate deploy</code> on first boot, then serves.
<code>mysql</code>	3306	MySQL 8.4 with the <code>tasktracker</code> database pre-created.
<code>redis</code>	6379	Cache + SSE pub/sub backplane.

The API waits on the MySQL and Redis healthchecks before starting, and the migration step retries until the database actually accepts connections — so the very first run is race-free even while MySQL is still initializing.

Once it's up

What	URL
API base	<code>http://localhost:3000</code>
Health probe	<code>http://localhost:3000/health</code>
Swagger UI	<code>http://localhost:3000/docs</code>
OpenAPI JSON	<code>http://localhost:3000/docs/openapi.json</code>

Frontend (optional bonus, second terminal)

```
cd frontend
npm install
npm run dev           # → http://localhost:5173
```

Register a new org (you become ADMIN), then explore Board / Projects / Users / Analytics. The notification bell updates live over SSE.

Cloud / local-node alternative. To run the API directly with `npm run dev`, copy `.env.example` (its defaults point at the compose MySQL/Redis) and edit `DATABASE_URL` / `REDIS_URL` to use any local or managed cloud instance (e.g. Aiven, Redis Cloud).

3 · API Documentation

Interactive, always-in-sync documentation ships with the API:

- **Swagger UI** — `/docs` (try every endpoint in the browser).
- **OpenAPI 3.0 spec** — `/docs/openapi.json` (importable into Postman/Insomnia).
- **Postman collection** — `docs/postman-collection.json` in the repo.

Consistent error shape

Every endpoint returns errors in one shape (Zod validation produces field-level messages):

```
{
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "dueDate: dueDate must be a future date",
  "details": [{ "field": "dueDate", "message": "dueDate must be a future date" }]
}
```

Endpoints

Method	Path	Description & access
Auth		
POST	<code>/auth/register</code>	Create a new org; caller becomes ADMIN.
POST	<code>/auth/login</code>	Login → access + refresh tokens.
POST	<code>/auth/refresh</code>	Rotate refresh token → new access + refresh pair.
POST	<code>/auth/logout</code>	Revoke a refresh token.
POST	<code>/auth/invite</code>	Invite a user into the org. <i>ADMIN</i>
GET	<code>/auth/me</code>	Current user from JWT.
Users		
GET	<code>/users</code>	List users in your org.
GET	<code>/users/:id</code>	Get one user.
PATCH	<code>/users/:id/role</code>	Change role; can't demote last ADMIN or self. <i>ADMIN</i>
DELETE	<code>/users/:id</code>	Delete user; reassigns their tasks/projects to caller. <i>ADMIN</i>
Projects		
GET	<code>/projects</code>	List projects.
GET	<code>/projects/:id</code>	Get one project.
POST	<code>/projects</code>	Create. <i>ADMIN, MANAGER</i>
PATCH	<code>/projects/:id</code>	Update. <i>ADMIN, MANAGER</i>

DELETE	<code>/projects/:id</code>	Delete (cascades to tasks). <i>ADMIN, MANAGER</i>
Tasks		
GET	<code>/tasks</code>	List — pagination (<code>page</code> , <code>limit</code>) + filter by status / priority / assignee. MEMBER sees only their assigned tasks.
GET	<code>/tasks/:id</code>	Get one task.
POST	<code>/tasks</code>	Create. <i>ADMIN, MANAGER</i>
PATCH	<code>/tasks/:id</code>	Update fields; MEMBER may edit only their own task and cannot reassign.
PATCH	<code>/tasks/:id/status</code>	Advance status via the enforced state machine. <i>Assignee or MANAGER/ADMIN</i>
DELETE	<code>/tasks/:id</code>	Delete. <i>ADMIN, MANAGER</i>
Notifications & Analytics		
GET	<code>/notifications</code>	List my notifications.
PATCH	<code>/notifications/:id/read</code>	Mark one read.
PATCH	<code>/notifications/read-all</code>	Mark all read.
GET	<code>/notifications/stream</code>	Real-time SSE stream (Bearer header or <code>?token=</code>).
GET	<code>/analytics/users</code>	Per-user overdue/open/done + avg completion + rank (window functions). <i>ADMIN, MANAGER</i>

Status state machine (server-enforced)

```

TODO → IN_PROGRESS → IN_REVIEW → DONE
  ↳ BLOCKED (reachable from any active state; DONE is terminal)

```

RBAC is enforced at the middleware layer (`requireAuth` + `requireRole`) on the routes — not inside controllers.

4 · Tradeoffs & Intentional Omissions

Conscious calls made under the time box, documented per the brief:

1. No rate limiting (intentional). Left out so the API can be exercised freely during review — load tests, seed scripts, and the integration tests all hammer the endpoints without being throttled. In production I'd add `express-rate-limit` backed by a Redis store: strict per-IP limits on the auth routes (brute-force defence) and looser per-user limits elsewhere.

2. Immediate hard delete, not a full two-stage soft delete. Deletes remove the row now — users reassign their tasks/projects to the actor first, projects cascade to their tasks, and notifications keep `taskId` via `SET NULL`. A `deletedAt` soft-delete only earns its cost once paired with a Trash/restore UI and a retention cron; without those it's pure overhead (every query needs `WHERE deletedAt IS NULL`, and MySQL can't do partial unique indexes for `(orgId, email)`). So the full soft-delete → Trash → retention / GDPR-anonymization flow was deferred.

3. JWT in `localStorage` + `Authorization: Bearer`, not `httpOnly` cookies. Consistent across REST and the fetch-based SSE stream (so the token never lands in a URL or log), but `localStorage` is readable by JavaScript and thus exposed to XSS. The hardening — moving the refresh token into an `httpOnly` + `Secure` + `SameSite` cookie (with CSRF protection) and issuing single-use, short-TTL tickets for the SSE stream — is documented as the next step rather than half-implemented.

5 • Improvements — Given More Time

1. **Two-stage soft delete** with a Trash UI bucket + cron-driven hard delete after a retention window, plus GDPR-compliant PII anonymization on right-to-be-forgotten requests.
2. **Message queue (RabbitMQ / Kafka) alongside Redis pub/sub.** Redis stays for low-latency real-time UI; the queue handles "must eventually run" work — email digests, due-date reminder cron, webhook delivery.
3. **Immutable audit log table** that survives even hard deletes — required for compliance.
4. **Per-user rate limiting** (`express-rate-limit` + Redis store).
5. **Harden token storage to `httpOnly` cookies** — move the refresh token to an `httpOnly` + `Secure` + `SameSite` cookie with CSRF protection, and issue single-use short-TTL tickets for the SSE stream instead of reusing the access token.
6. **Distributed tracing** (OpenTelemetry → Jaeger) so request paths through middleware/service/DB are observable.
7. **Drag-and-drop kanban** on the task board (currently a status dropdown per card).
8. **Comments, attachments, full-text search** (Meilisearch) on tasks.

Already delivered beyond the core: analytics endpoint (SQL window functions), real-time SSE notifications (single shared subscriber — constant Redis connections), a full React task-board frontend, and Jest integration tests for two critical flows (refresh-token rotation and the status state machine).